CURSO 2026 - 2027

MANUAL DE REFERENCIA DEL ALUMNADO

UT02 · MongoDB Avanzado y Atlas

CICLO FORMATIVO

Desarrollo de Aplicaciones Web (DAW) · 2º curso

MÓDULO

Tecnologías Web Actuales (Optativa)

UNIDAD DE TRABAJO

UT02 · MongoDB Avanzado y Atlas

Miguel Ángel Rubiano Barros
I.E.S. Suárez de Figueroa — Departamento de Informática y Comunicaciones

Índice de contenidos

Capítulo 1 · MongoDB Atlas: tu primer clúster en la nube 3

Capítulo 2 · Framework de agregación 7

Capítulo 3 · Actividad 3: práctica guiada de agregación 12

Capítulo 4 · Modelado de datos avanzado 16

Capítulo 5 · Validación de esquemas y seguridad básica 20

Capítulo 6 · Cierre de la unidad y próximos pasos 24

Capítulo 1 · MongoDB Atlas: tu primer clúster en la nube

Pasamos de un servidor MongoDB instalado en nuestro propio equipo a un clúster gestionado en la nube. Es el primer paso de esta unidad, y la base sobre la que trabajaremos durante toda la UT02.

1.1 ¿Qué es MongoDB Atlas?

MongoDB Atlas es el servicio de base de datos como servicio (*DBaaS*, Database as a Service) oficial de MongoDB: en lugar de instalar y administrar tú mismo el servidor (como hicimos en la UT01), Atlas se encarga de aprovisionar, actualizar, vigilar y proteger el servidor por ti, en la infraestructura en la nube de tu elección (AWS, Google Cloud o Azure).

- **Sin instalación local**: el clúster vive en la nube; te conectas a él desde mongosh, Compass o tu aplicación, igual que a un servidor local, pero a través de internet.
- **Copias de seguridad gestionadas**: Atlas puede generar copias de seguridad automáticas y restaurar a un punto concreto en el tiempo, sin que tengas que ejecutar mongodump manualmente.
- **Escalado y alta disponibilidad**: un clúster de Atlas puede configurarse con réplicas (para tolerar fallos) y sharding (para repartir datos entre varios servidores) con unos pocos clics.
- **Nivel gratuito (M0)**: Atlas ofrece un clúster compartido gratuito, perfecto para aprender, hacer prácticas y proyectos pequeños — el que usaremos durante todo este manual.

Idea clave. Todo lo que aprendiste en la UT01 —CRUD, índices, agregación— funciona exactamente igual en Atlas: el lenguaje de consultas (MQL) no cambia. Lo único que cambia es **dónde** vive el servidor y **cómo** te conectas a él.

1.2 Creación de una cuenta y un clúster gratuito (M0)

El primer paso es crear una cuenta en **cloud.mongodb.com** (puede hacerse con un correo electrónico o vinculando una cuenta de Google/GitHub). Una vez dentro, Atlas organiza el trabajo en **organizaciones** y **proyectos**: una organización puede contener varios proyectos, y cada proyecto puede contener uno o varios clústeres.

- Crea un **proyecto** nuevo (por ejemplo, "Tecnologías Web Actuales").
- Dentro del proyecto, pulsa **Create a deployment** (o **Build a Database**) y elige el plan **M0 Free** — gratuito de por vida, sin necesidad de tarjeta de crédito.
- Elige un **proveedor de nube** (AWS, Google Cloud o Azure) y una **región** cercana (por ejemplo, Europa — Irlanda o Frankfurt).
- Dale un nombre al clúster (por ejemplo, Cluster0, el nombre que propone Atlas por defecto) y confirma la creación.

Para saber más. El clúster M0 tiene límites pensados para aprendizaje y pruebas: 512 MB de almacenamiento y recursos compartidos con otros clústeres M0. Es más que suficiente para

tienda_online y para todos los ejercicios de este manual, pero no es apto para un proyecto en producción con tráfico real.

1.3 Usuarios de base de datos

Atlas separa dos tipos de credenciales: las de tu **cuenta de Atlas** (con las que entras al panel de administración web) y los **usuarios de base de datos** (con los que tus aplicaciones y herramientas, como mongosh, se conectan al propio clúster). Al crear el clúster, Atlas te pedirá crear el primer usuario de base de datos:

- **Nombre de usuario y contraseña**: el método más simple y el que usaremos en este curso. Atlas puede generar una contraseña segura automáticamente.
- **Rol de acceso**: por defecto, Atlas asigna el rol **Read and write to any database**, adecuado para desarrollo y prácticas. En el Capítulo 5 veremos cómo crear usuarios con roles más restringidos.

Atención. Guarda la contraseña del usuario de base de datos en un lugar seguro en el momento de crearla: Atlas no la vuelve a mostrar en texto plano después (aunque siempre puedes editarla y generar una nueva).

1.4 Lista de acceso de red (IP Access List)

Por seguridad, Atlas **bloquea por defecto todas las conexiones** a tu clúster. Antes de poder conectarte, tienes que añadir tu dirección IP (o un rango de IPs) a la **lista de acceso de red** (*IP Access List*), en la sección **Network Access** del panel de Atlas.

- **Add Current IP Address**: añade automáticamente la IP pública desde la que te estás conectando en ese momento.
- **Allow Access from Anywhere** (0.0.0.0/0): abre el acceso a cualquier IP. Es cómodo para practicar desde distintos equipos (casa, instituto), pero **nunca debe usarse así en un entorno de producción real**, ya que elimina esta capa de protección.

Atención. Para las prácticas de este curso, usar "Allow Access from Anywhere" es razonable (no hay datos reales en juego), pero es importante que entiendas que es una decisión de seguridad relajada de forma deliberada para el aula, no la práctica recomendada en un proyecto real.

1.5 Conexión remota con mongosh

Con el usuario de base de datos creado y la IP autorizada, ya podemos conectarnos. Desde el panel de Atlas, el botón **Connect** del clúster ofrece la cadena de conexión (*connection string*) necesaria, con el prefijo `mongodb+srv://` en lugar del `mongodb://localhost` que usábamos en la UT01:

```
mongosh "mongodb+srv://cluster0.abcde.mongodb.net/" --apiVersion 1 --username
miUsuario
// mongosh pedirá la contraseña de forma interactiva (no queda guardada en el
historial)
```

Conexión	Prefijo	Dónde se usa
`Servidor local (UT01)`	mongodb://	localhost, un único servidor
`Clúster de Atlas (UT02)`	mongodb+srv://	DNS gestionado por Atlas, resuelve automáticamente a los nodos del clúster

Para saber más. El prefijo `mongodb+srv://` usa un registro DNS especial (SRV) que resuelve automáticamente a todos los nodos del clúster, sin que tengas que conocer ni escribir sus direcciones una por una. Es la forma recomendada de conectar con cualquier clúster de Atlas, tenga una o varias réplicas.

Una vez conectado, el prompt de mongosh es idéntico al que ya conoces de la UT01: los mismos comandos (`show dbs`, `use`, `db.coleccion.find()`...) funcionan exactamente igual, solo que ahora se ejecutan contra el clúster remoto en lugar de tu servidor local.

1.6 Importando tienda_online en Atlas

Igual que en la UT01 (Capítulo 8), usamos `mongoimport` para cargar datos, pero ahora apuntando a la cadena de conexión del clúster en lugar de a `localhost`. Para la UT02 trabajaremos con una versión **ampliada** del dataset de la tienda, que añade una colección `clientes` relacionada con `productos` a través del campo `ultima_compra`:

```
$MONGO_URI="mongodb+srv://usuario:contraseña@cluster0.mongodb.net/tienda_online"

mongoimport "$MONGO_URI" --collection=productos \
  --file=productos_ampliado.json --jsonArray

mongoimport "$MONGO_URI" --collection=clientes \
  --file=clientes.json --jsonArray
```

Atención. Cuando la cadena de conexión incluye usuario y contraseña directamente (en lugar de pedirla mongosh de forma interactiva), cualquier carácter especial de la contraseña (@, /, :, etc.) debe codificarse con *percent-encoding*. Si tu contraseña generada por Atlas incluye alguno de estos caracteres, es más sencillo regenerarla desde el panel pidiendo una sin símbolos especiales.

Tras importar, comprobamos en mongosh que todo ha llegado correctamente, igual que hacíamos en la UT01:

```
use tienda_online
show collections
// -> clientes, productos

db.productos.countDocuments()
db.clientes.countDocuments()
```

```
db.clientes.findOne()
```

1.7 Compass y Charts contra un clúster de Atlas

MongoDB Compass, que ya conoces de la UT01, se conecta a un clúster de Atlas exactamente igual que a un servidor local: basta con pegar la misma cadena de conexión `mongodb+srv://` en la pantalla de conexión de Compass.

Atlas añade además **MongoDB Charts**, una herramienta de visualización de datos integrada: permite construir gráficos y paneles (*dashboards*) directamente a partir de las colecciones del clúster, sin escribir código. La retomaremos de forma introductoria al hablar de analítica de datos en el Capítulo 5.

> **Idea clave.** A partir de aquí, cada vez que este manual muestre un comando de mongosh, se asume que ya estás conectado al clúster de Atlas (salvo que se indique explícitamente lo contrario). No hace falta repetir la cadena de conexión en cada ejemplo.

Capítulo 2 · Framework de agregación

Cómo resumir y transformar datos con el pipeline de agregación: la herramienta que va mucho más allá de lo que permite find(), y que usaremos constantemente durante el resto de esta unidad sobre el clúster de Atlas dado de alta en el Capítulo 1.

2.1 El framework de agregación

La **agregación** es el proceso de calcular resultados resumidos (totales, promedios, conteos, agrupaciones…) a partir de una colección de documentos. MongoDB ofrece para ello el **pipeline de agregación**: una secuencia de etapas encadenadas, donde la salida de cada etapa es la entrada de la siguiente.

Esta idea es muy similar a la cláusula `GROUP BY` de SQL, pero bastante más flexible, ya que permite encadenar libremente muchas más transformaciones aparte de la agrupación. Se invoca con el método `aggregate()` sobre una colección, pasándole un **array de etapas**:

```
db.coleccion.aggregate([
  { $etapa1: { ... } },
  { $etapa2: { ... } },
  // ...
])
```

Es la alternativa recomendada frente a Map-Reduce (una técnica más antigua y compleja) para la inmensa mayoría de los casos de agregación de datos.

2.2 Etapas principales del pipeline

Etapa	Función
`$match`	Filtra documentos según una condición (equivalente a find() con criterio)
`$group`	Agrupa documentos por una llave y calcula resultados agregados para cada grupo
`$project`	Reestructura cada documento: selecciona campos o calcula campos nuevos
`$sort`	Ordena el resultado según uno o varios campos, ascendente o descendente
`$limit / $skip`	Limitan el número de documentos en la salida, o saltan un número de documentos (paginación)
`$unwind`	Desenrolla un array, creando un documento separado por cada elemento del array
`$lookup`	Combina datos de otra colección (equivalente a un JOIN de SQL)
`$facet`	Ejecuta varios sub-pipelines de agregación en una sola operación
`$out`	Escribe el resultado de la agregación en una colección nueva

Idea clave. No existe un límite fijo en el número de etapas que se pueden combinar: se encadenan tantas como necesite el análisis. **El orden importa**: cada etapa trabaja sobre el resultado que le entrega la anterior, así que, por ejemplo, filtrar con `$match` **antes** de un `$group` (en lugar de después) reduce el número de documentos que hay que agrupar y mejora mucho el rendimiento.

2.3 $match y $group: el dúo fundamental

`$match` filtra documentos exactamente igual que el primer argumento de `find()`. `$group` agrupa los documentos que le llegan según el valor de un campo (su `_id` de grupo) y calcula, para cada grupo, uno o varios valores agregados mediante **operadores de acumulación**.

```
db.productos.aggregate([
  { $match: { categoria: "electronica" } },
  { $group: {
      _id: '$categoria',
      totalProductos: { $sum: 1 },
      totalStock: { $sum: '$stock' },
      precioPromedio: { $avg: '$precio' }
  } }
])
```

Primero filtramos (solo electrónica) y después agrupamos: el total de productos, la suma de stock y el precio medio se calculan ya sobre el subconjunto filtrado.

Atención. Dentro de una etapa de agregación, cuando queremos referirnos **al valor de un campo** del documento (y no a un texto literal) escribimos su nombre precedido de `$`, entre comillas: `'$categoria'`, `'$stock'`. Es una fuente de errores muy habitual al empezar: olvidar el `$` hace que MongoDB interprete el texto como una cadena literal en lugar de como una referencia a un campo.

2.4 Operadores de acumulación

Los operadores de acumulación son expresiones especiales que solo se pueden usar dentro de `$group` (no están disponibles, por ejemplo, dentro de un `find()` normal), y que calculan un valor agregado a partir de los documentos de cada grupo.

Operador	Función
`$sum`	Suma los valores (o cuenta documentos, si se usa $sum: 1)
`$avg`	Calcula el promedio (media) de los valores numéricos
`$min / $max`	Obtiene el valor mínimo o máximo de un campo
`$push`	Agrega valores a un array (incluye duplicados)
`$addToSet`	Agrega valores a un array solo si no están ya (evita duplicados)
`$stdDevPop`	Calcula la desviación estándar poblacional de un campo numérico
`$first / $last`	Toma el primer o el último valor de un grupo (requiere un $sort previo)

- **$sum: 1** es el patrón más habitual para **contar documentos** dentro de cada grupo (cada documento "suma 1" al contador del grupo al que pertenece).
- **$push vs $addToSet**: `$push` lista todos los valores, permitiendo duplicados; `$addToSet` solo agrega valores únicos.
- **$first / $last** solo tienen sentido si el pipeline incluye un `$sort` **antes** del `$group`, ya que dependen del orden en que "llegan" los documentos a cada grupo.
- Los acumuladores **solo están disponibles dentro de `$group`** (o de etapas de agregación relacionadas), no se pueden usar en un `find()` normal.
- `_id: null` agrupa **todos** los documentos en un único grupo, útil cuando queremos una estadística global de toda la colección en lugar de una por cada valor de un campo.

Para saber más. Además de los operadores anteriores, MongoDB ofrece otros más especializados que conviene conocer de nombre, aunque no los usemos a diario: `$mergeObjects` combina varios subdocumentos en uno solo, y variantes como `$firstN` / `$lastN` (en versiones recientes) permiten obtener varios elementos ordenados de un grupo a la vez, en lugar de uno solo. Conviene revisar siempre la documentación oficial para conocer el catálogo completo.

2.5 $project: reestructurar documentos

`$project` selecciona, renombra o calcula campos nuevos en cada documento de salida, de forma parecida a la proyección de `find()`, pero con muchas más posibilidades (puede calcular expresiones, no solo incluir o excluir campos tal cual).

```
db.productos.aggregate([
  { $project: {
      nombre: 1,
      precioConIva: { $multiply: ['$precio', 1.21] },
      _id: 0
  } }
])
```

2.6 $lookup: uniones entre colecciones

`$lookup` es el operador de agregación que permite realizar **uniones entre colecciones**, de forma similar a un `JOIN` de SQL. Su sintaxis básica es:

```
{ $lookup: {
    from: 'coleccionAUnir',
    localField: 'campoLocalObjetivo',
    foreignField: 'campoDeColeccionAUnirObjetivo',
    as: 'resultado'
} }
```

Ejemplo práctico: mostrar los nombres de los clientes junto con los datos del producto de su última compra, cruzando la colección `clientes` con la colección `productos`:

```
db.clientes.aggregate([
  {
    $lookup: {
      from: 'productos',
      localField: 'ultima_compra',
      foreignField: 'nombre',
      as: 'producto_comprado'
    }
  },
  { $project: { nombre: 1, 'producto_comprado.precio': 1 } }
])
```

Cada documento del resultado tendrá un nuevo array (`producto_comprado`, en este ejemplo) con los documentos coincidentes de la otra colección, siguiendo la relación `localField = foreignField`. Es muy habitual encadenar un `$unwind` justo después de un `$lookup` para "desenrollar" ese array y acceder a sus campos de forma más cómoda, como veremos en la práctica guiada del Capítulo 3.

2.7 $facet: múltiples pipelines en una sola operación

`$facet` ejecuta **varios sub-pipelines de agregación en paralelo**, todos ellos sobre el mismo conjunto de documentos de entrada, y devuelve sus resultados agrupados en un único documento. Es ideal para construir *dashboards* que necesitan varias métricas distintas calculadas a partir de la misma consulta base.

```
db.productos.aggregate([
  { $facet: {
      top_precios: [
        { $sort: { precio: -1 } },
        { $limit: 5 }
      ],
      por_categoria: [
        { $group: { _id: '$categoria', total: { $sum: 1 } } }
      ]
  } }
])
```

En una sola llamada obtenemos, a la vez, los 5 productos más caros y el total de productos por categoría.

Etapa	Función
`$match`	Filtrar documentos por un criterio
`$group`	Agrupar documentos y calcular agregados

Etapa	Función
`$project`	Seleccionar o transformar campos
`$sort`	Ordenar resultados según campos
`$limit / $skip`	Limitar o paginar resultados
`$unwind`	Expande documentos por elementos de un array
`$lookup`	Combinar documentos de otra colección (join)
`$out`	Escribir resultados en una colección
`$facet`	Ejecuta varios pipelines de agregación

Para saber más. Cada etapa del pipeline procesa hasta 100 MB de datos en memoria. Si una agregación necesita más (por ejemplo, al agrupar colecciones muy grandes), se puede habilitar la opción `allowDiskUse: true` para permitir el uso de disco, aunque esto resulta más lento. El resultado de una agregación también está sujeto al límite de tamaño de documento de MongoDB (16 MB), salvo que se use `$out` para escribirlo en una colección.

Idea clave. En el panel de Atlas, la pestaña **Aggregation** del explorador de colecciones ofrece un constructor visual de pipelines: permite añadir etapas una a una y ver el resultado parcial después de cada una, lo que resulta muy útil para depurar un pipeline paso a paso antes de trasladarlo a mongosh.